AquaGuard

Real-Time Detection of Early Drowning and Distress Using Computer Vision

Project Documentation

Item	Details
Program	Digital Egypt Pioneers Initiative (DEPI) - AI & Data Science, Microsoft Machine Learning Engineer track
Team	Omar Sayed Darder Malak Gamal Abdelkarim Sayed Mohamed Hazem Elsherbiny Habiba Mohammed Adley Nourhan Adel Mohamed Elhady

1- Planning and Management

1.1 Project Proposal

1.1.1 Overview

Drowning is a major cause of accidental death. The World Health Organization estimates about 300,000 deaths a year, around 92% of them in low- and middle-income countries. Lifeguards are the main protection at pools and beaches, but watching water for hours is tiring, and a person in trouble often makes little noise or movement. Glare, waves and crowds make it harder still.

AquaGuard is a computer vision system that reads a camera feed, finds and follows swimmers, checks their movement over a few seconds, and alerts a human operator when the behaviour looks like early drowning or distress. It supports lifeguards and does not replace them. The pipeline has video input (OpenCV), person detection (YOLO with PyTorch), tracking and pose estimation, behaviour analysis, and an alert and dashboard. MLflow and Azure are used to track experiments and store models.

1.1.2 Objectives

- By 5 Nov, build an annotated dataset of at least 3,000 frames and 100 behaviour clips.
- By 19 Nov, train a detector with mAP@0.5 of at least 0.85 on a test set that includes glare and crowded scenes, and log all final runs in MLflow.
- By 26 Nov, detect distress with recall of at least 90% and precision of at least 75% on test clips.
- By 26 Nov, run at 15 FPS or more on the demo laptop, with an alert within 5 seconds of the distress being confirmed.
- By 4 Dec, submit all DEPI deliverables on GitHub.

These targets are ours and will be reviewed after the first baseline results.

1.1.3 Scope

In scope

- Swimming pools (indoor and outdoor), with beach and open-water footage as a secondary test.
- Daylight and well-lit conditions, with moderate glare.
- A demo on a laptop/PC using a webcam, a camera stream or recorded video.
- Alerts and a dashboard for a human operator. Azure free/student tier for tracking and storage.

Out of scope

- Night, rain, fog, dark or muddy water, strong waves, and underwater cameras.
- Edge devices (Jetson, Raspberry Pi), many cameras, drones and physical alarms.
- Identifying people, automatic rescue, contact with emergency services, and recording real emergencies.

1.2 Project Plan

1.2.1 Timeline

The work runs from 14 October to 4 December 2026 in one-week sprints. Week labels show the Sunday each week starts. **D** marks a **DEPI submission** and **M** an **internal milestone**. We stop adding new features on 26 November and use the last week for testing and documentation.

Table 1. Gantt chart

Workstream	W0 14-16 Oct	W1 18 Oct	W2 25 Oct	W3 1 Nov	W4 8 Nov	W5 15 Nov	W6 22 Nov	W7 29 Nov
Planning, literature review, requirements	D1							
System analysis and design				D2				
Data collection and annotation				M1				
Model experimentation						M2		
MLOps (Azure, MLflow)								
Integration and deployment							M3	
Implementation submission								D3
Testing								
Final documents and presentation								D4

1.2.2 Milestones and Deliverables

Table 2. Milestones and deliverables

Date	Milestone	Deliverables
16 Oct	D1 Phase 1 submission	Project Planning & Management, Literature Review, Requirements Gathering
5 Nov	M1 Dataset ready	Annotated dataset, labelling guide, train/validation/test split
6 Nov	D2 System Analysis & Design	Architecture and UML diagrams, dashboard wireframes, technology stack, deployment and component diagrams
19 Nov	M2 Final model	Trained and registered model in MLflow, model card, test results
26 Nov	M3 Integrated demo	Working pipeline with alerts and dashboard on the demo laptop
30 Nov	D3 Implementation	Source code and README on GitHub, runnable demo package
4 Dec	D4 Final Presentation, Testing & Reports	Test plan and results, bug reports, user manual, technical documentation, presentation, demo video, infographic

1.2.3 Resource Allocation

- Software: Python, PyTorch, YOLO (Ultralytics), OpenCV, a ByteTrack-style tracker, Streamlit or FastAPI for the dashboard.
- MLOps: MLflow for tracking and model registry, Azure Machine Learning workspace and Blob Storage (free/student tier), GitHub for code and CI, Hugging Face Hub for model cards.

- Compute: Google Colab or Kaggle free GPUs for training, since GPU quota on a free Azure tier is usually not available (we will check in week 1). The demo laptop/PC is used for inference.
- Annotation: CVAT or Label Studio.
- Data: public or licensed footage, and recordings of adult volunteers who agreed, supervised by a lifeguard.
- People: five team members working in one-week sprints (roles in Section 3).

1.3 Task Assignment and Roles

- Omar: team lead, planning, architecture and behaviour analysis.
- Malak: data engineering and annotation.
- Mohamed: model training and optimisation.
- Habiba: Azure and MLflow pipeline and CI.
- Nourhan: system integration, dashboard and documentation.

Table 3. RACI matrix (R = responsible, A = accountable, C = consulted, I = informed)

Task	Omar	Malak	Mohamed	Habiba	Nourhan
Project management and architecture	**A/R**	C	C	C	C
Data collection and annotation	I	**A/R**	C	R	R
Preprocessing and dataset versions	C	**A/R**	C	R	I
Model training and optimisation	**A**	C	R	C	I
Behaviour analysis logic	**A/R**	I	R	I	C
Azure and MLflow pipeline	**A**	I	C	R	C
System integration and dashboard	**A**	I	C	C	R
Testing and validation	**A**	C	R	R	R
Documentation and presentation	C	R	R	R	**A/R**

1.4 Risk Assessment and Mitigation Plan

Severity combines how likely a risk is and how much damage it would cause. It will be reviewed at the end of each sprint.

Table 4. Risks and mitigation

Risk	Severity	Mitigation	Fallback
Very little real drowning footage	Critical	Describe distress as visible behaviours. Use public footage plus supervised recordings of simulated distress. Use augmentation. Report which results come from simulated data.	Claim only "early distress signs" and validate a rule-based detector on recorded clips.
Glare and reflections on the water	Critical	Collect footage in different lighting. Add glare and brightness augmentation. Evaluate each condition separately.	Limit the demo to controlled lighting.
Occlusion and partial submersion	High	Use a tracker that tolerates short gaps and smooth results over time.	Show a "lost track" warning for the operator.
Missed events and false alarms	High	Choose the threshold for high recall. Use a confirmation window and a cooldown. Measure false alarms per hour.	Use a silent highlight first and sound the alarm only if the condition continues.
Low speed on the laptop/PC	High	Use a small model, a smaller input size and ONNX export. Measure FPS early.	Process every n-th frame or demo on a GPU laptop.

Risk	Severity	Mitigation	Fallback
Privacy and volunteer safety	High	Use only public footage or adults who agreed. Blur or avoid faces. Keep a lifeguard present at recordings. Ask the instructors for approval.	Use public recorded footage only.
Azure free/student limits	High	Use Azure only for tracking, storage and registry. Train on Colab or Kaggle. Set budget alerts.	Run MLflow locally with the same code.
Short schedule	High	Stop new features on 26 Nov. Start MLOps and integration before the final model. Check milestones every week.	Drop optional parts first (learned behaviour model, open-water tests).

1.5 Key Performance Indicators (KPIs)

Table 5. KPIs

KPI	Target	How it is measured
Recall (sensitivity)	At least 90%	Distress test clips
Precision	At least 75%	Distress clips and long normal footage
mAP@0.5 (detection)	At least 0.85	Held-out test scenes
Inference speed	At least 15 FPS (about 70 ms per frame)	Benchmark on the demo laptop/PC
Alert response time	5 seconds or less	Time-stamped logs on replayed clips
False alarm rate	1 or less per camera-hour	Long normal videos
System uptime	At least 99% over a 4-hour run	Long-run test
User adoption / usability	At least 4 out of 5 from at least 5 test users	Short questionnaire after the demo
Milestone adherence	At least 85% on time	Checked at every sprint review
Sprint completion rate	At least 80% of planned tasks	End of each sprint

2- Requirements Gathering

2.1 Stakeholder Analysis

The lifeguard/operator is the main user; the others set limits on data, privacy and delivery.

Stakeholder	Key needs
Lifeguard / operator	Alert within 5 s; at most 1 false alarm per camera-hour; clear flagged swimmer; "lost track" warning
Pool or beach management	Reliable system (99% uptime), daylight use, no identification of people
Swimmers and volunteers	Safety, agreement before filming, faces blurred, no real emergencies recorded
Team, instructors, DEPI	Feasible scope, free tools, deliverables on GitHub on time
Test users (5+)	An understandable dashboard (at least 4 out of 5)

2.2 User Stories & Use Cases

Priority: Must / Should / Could.

2.2.1 User Stories

ID	As a...	I want to...	Priority
US1	Operator	see live video with each swimmer tracked	Must
US2	Operator	get an alert within 5 s of confirmed distress	Must
US3	Operator	see a "lost track" warning when a swimmer is hidden	Must
US4	Operator	hear the alarm only after a confirmation window, with a silent highlight first	Must
US5	Operator	see an early "watch" level before "alert"	Should
US6	Operator	use a webcam, stream or recorded video, and review time-stamped logs	Must
US7	Team member	annotate data and log runs in MLflow	Must
US8	Volunteer	be filmed only with my agreement and with my face blurred	Must

2.2.2 Use Cases

ID	Use case	Main flow
UC1	Monitor the water	Operator picks a video source; the system detects and tracks swimmers; the dashboard shows the feed.
UC2	Receive a distress alert	The system checks posture and movement over a few seconds, confirms the distress, and alerts within 5 s; the operator responds.

ID	Use case	Main flow
UC3	Handle a lost track	A swimmer is hidden; the tracker bridges short gaps; a "lost track" warning shows if the gap is too long.

2.3 Functional Requirements

Pipeline: video input, detection, tracking and pose, behaviour analysis, alert and dashboard, with MLOps.

ID	Requirement	Priority
FR1	Read video from a webcam, camera stream or recorded file (OpenCV).	Must
FR2	Detect swimmers in each frame (YOLO with PyTorch), in daylight with moderate glare and in crowds.	Must
FR3	Track each swimmer with an ID through short gaps (ByteTrack-style) and show a "lost track" warning.	Must
FR4	Estimate the pose of each tracked swimmer.	Must
FR5	Analyse posture and movement over a few seconds to flag early drowning or distress (rule-based first).	Must
FR6	Use a confirmation window and cooldown to limit false alarms, with an early "watch" level before "alert".	Must
FR7	Alert the operator within 5 seconds of confirmed distress.	Must
FR8	Dashboard (Streamlit or FastAPI) showing the live feed, track IDs, status, alerts and time-stamped logs.	Must
FR9	Build the annotated dataset (3,000 frames, 100 clips); log runs and register the final model in MLflow.	Must
FR10	Publish source code, README and a runnable demo package on GitHub.	Must

2.4 Non-functional Requirements

Targets are the team's own and will be reviewed after the first baseline results.

ID	Category	Requirement and target
NFR1	Speed	At least 15 FPS (about 70 ms per frame) on the demo laptop/PC
NFR2	Response	Alert within 5 seconds or less
NFR3	Detection	mAP@0.5 of at least 0.85

ID	Category	Requirement and target
NFR4	Accuracy	Distress recall at least 90%, precision at least 75%
NFR5	Reliability	1 or fewer false alarms per camera-hour
NFR6	Reliability	At least 99% uptime over a 4-hour run
NFR7	Privacy	No identification of people; faces blurred or avoided; no real emergencies recorded
NFR8	Data ethics	Only public footage or consenting adult volunteers, with a lifeguard present and instructor approval
NFR9	Usability	At least 4 out of 5 from at least 5 test users
NFR10	Portability and quality	Runs on a laptop/PC within free tiers; modular, commented code with a README; input validation and error handling

2.5 Scope Limits and Traceability

Out of scope: night, rain, fog, dark or muddy water, strong waves, underwater cameras; edge devices, many cameras, drones, physical alarms; identifying people, automatic rescue, calling emergency services, recording real emergencies.

Objective	Date	Requirements
1. Annotated dataset	5 Nov	FR9
2. Detector with mAP@0.5 of at least 0.85	19 Nov	FR2, FR9, NFR3
3. Distress detection (recall 90%, precision 75%)	26 Nov	FR3 to FR6, NFR4, NFR5
4. At least 15 FPS, alert within 5 s	26 Nov	FR7, FR8, NFR1, NFR2
5. Deliverables on GitHub	4 Dec	FR10

3- Literature Review

3.1 Introduction

Drowning causes about 300,000 deaths a year worldwide [1]. This review looks at earlier work on detecting drowning with cameras and computer vision, to see what has been tried, what results were reported and what is still missing. For most papers we could read only the abstract and publication page, so the numbers below are those reported by the authors. We did not repeat their experiments.

3.2 Related Work

Drowning is often silent. A person may be upright and bobbing in the water, press the arms down on the surface, and be unable to call for help [2]. This is why AquaGuard looks at posture and movement over a few seconds, and not only at whether a person is in the picture.

Most drowning detection papers use YOLO, a detector that finds objects with a single network and runs at 45 frames per second in its original form [3]. Proctor ran YOLOv8 on an Oak-D Lite edge camera and found real-time detection feasible [4]. Amer et al. ran YOLOv11 on a Raspberry Pi 5 and report mAP@50 of about 0.98 and recall of 92.6%, but on a small simulated dataset (1,519 images after augmentation), with weaker results for partly submerged and distant swimmers and only 3 to 4 frames per second [5]. Zeng et al. improved YOLOv5 and added a "transition" label so that an alert can be raised before the fully drowned state [6]. Alzaabi et al. built a dataset of 69,512 underwater frames and report 98.3% precision at 22 ms per frame with YOLOv8n, but the abstract gives no recall or results on unseen pools [7].

Other approaches exist. He et al. use an unsupervised convolutional autoencoder on an edge device, so labelled drowning footage is not needed [8]. Delgado et al. combine detection with classifiers and report 98.38% accuracy with a Pyramid Vision Transformer, on 3,712 images created through augmentation [9].

Drowning develops over time, so a swimmer has to be followed from frame to frame. ByteTrack keeps low-confidence detections to recover partly hidden objects and reaches 80.3 MOTA on MOT17 at 30 FPS [10]. This suits swimmers hidden by water and splashes. The abstracts we read mostly describe single-frame detection or classification.

SightBit is a commercial system that analyses beach camera video to flag swimmers in distress. It is introduced alongside human lifeguards, and it gives no public accuracy figures [11].

3.3 Gaps and How AquaGuard Responds

- Datasets are small, simulated or enlarged by augmentation. We will split by video or scene, and report results with and without simulated clips.
- Precision and accuracy are reported more often than recall and false alarms. We will use recall as the main measure and report false alarms per camera-hour.
- Most work looks at single frames. We will track each swimmer and use a confirmation window of a few seconds before alerting.
- Edge hardware can be slow. We will run the demo on a laptop/PC and measure FPS and latency on it.
- Pools are covered more than open water. We will use the pool as the main case and report open-water results separately.

3.4 Suggested Improvements

- Add an early "watch" level before the "alert" level, similar to the transition label in [6].
- Try a learned temporal model (1D-CNN or LSTM) once the rule-based version works.

- Collect more footage with crowds, glare and different camera angles, and test on a pool not used in training.

3.5 References

[1] World Health Organization. Drowning (fact sheet). https://www.who.int/news-room/fact-sheets/detail/drowning

[2] Royal Life Saving Society UK. Silent drowning: what's it all about. https://www.rlss.org.uk/News/silent-drowning-whats-it-all-about

[3] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. arXiv:1506.02640. https://arxiv.org/abs/1506.02640

[4] F. Proctor. Real Time Drowning Detection System Using Machine Vision and Learning. University of Southern Queensland, 2023. https://sear.unisq.edu.au/52993/

[5] D. A. Amer et al. Intelligent eyes on water: YOLOv11-based real-time drowning detection system. The Journal of Supercomputing, vol. 81, article 1242, 2025. https://link.springer.com/article/10.1007/s11227-025-07732-7

[6] X. Zeng, W. Xu, X. Zhang. DrownACB-YOLO: an Improved YOLO for Drowning Detection in Swimming Pools. Journal of Donghua University (English Edition), vol. 42, no. 4, 2025. https://journal.hep.com.cn/jdue/EN/10.19884/j.1672-5220.202406015

[7] H. Alzaabi, S. Alzaabi, S. Kohail. Multi-swimmer Drowning Detection Using a Custom Annotated Underwater Dataset and Real-Time AI. Lecture Notes in Computer Science, vol. 16168, Springer, 2026. https://zuscholars.zu.ac.ae/works/7734

[8] X. He, F. Yuan, T. Liu, Y. Zhu. A video system based on convolutional autoencoder for drowning detection. Neural Computing and Applications, vol. 35, no. 21, 2023. https://link.springer.com/10.1007/s00521-023-08526-9

[9] J. Delgado, S. Leon, W. Ticona. Model for the Automatic Detection of People at Risk of Drowning in Swimming Pools using Computer Vision. Lecture Notes in Networks and Systems, vol. 1901, Springer, 2026. https://link.springer.com/chapter/10.1007/978-3-032-22236-7_12

[10] Y. Zhang et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. arXiv:2110.06864, 2021. https://arxiv.org/abs/2110.06864

[11] SightBit AI lifeguard, reported in "Israel's AI lifeguard set to save swimmers on UAE beaches", ISRAEL21c. https://archive.israel21c.org/israels-ai-lifeguard-set-to-save-swimmers-on-uae-beaches/